

Task 1: Host a simple webpage on AWS

Prepared by Fatima Ezzahrae Ezzouzi

Cloud Programming (DLBSEPCP01_E)

Field of Study– Bachelor of Science in Applied Artificial Intelligence

Matriculation Number– 9217425

Tutor's Name– Ahmet Nasri

February 27, 2026

1. Introduction
 - 1.1 Project Overview
 - 1.2 Objectives
2. Project Idea and Concept
 - 2.1 Project Idea
 - 2.2 Concept and Design Principles
3. Architecture Design
 - 3.1 System Architecture Overview
 - 3.2 Architecture Components
 - 3.3 Architecture Workflow
4. Technologies Used
 - 4.1 Amazon Web Services (AWS)
 - 4.2 Terraform (Infrastructure as Code)
 - 4.3 Supporting Tools
5. Terraform Project Structure
 - 5.1 File Structure Overview
 - 5.2 provider.tf
 - 5.3 s3.tf
 - 5.4 cloudfront.tf
 - 5.5 variables.tf
 - 5.6 terraform.tfvars
 - 5.7 outputs.tf
6. Implementation and Deployment Process
 - 6.1 Terraform Initialization
 - 6.2 Planning and Validation
 - 6.3 Infrastructure Provisioning
 - 6.4 Website Deployment
7. Testing and Validation
8. Improvements from Phase 1 and Phase 2
9. Challenges and Solutions
10. Results and Final Evaluation
11. Conclusion

1. Introduction

1.1 Project Overview

This project presents the design and deployment of a static website hosted on Amazon Web Services (AWS) using Infrastructure as Code (IaC). The objective is to demonstrate how cloud infrastructure can be automated, secured, and optimized using Terraform.

The system integrates AWS Identity and Access Management (IAM), Amazon S3, and Amazon CloudFront to build a scalable, secure, and globally accessible architecture.

1.2 Objectives

- Automate infrastructure deployment using Terraform
- Host a static website using Amazon S3
- Improve global availability using CloudFront
- Apply security best practices using IAM
- Optimize infrastructure after feedback from Phase 1 and Phase 2

2. Project Idea and Concept

2.1 Project Idea

The idea behind this project is to replace manual cloud configuration with an automated, repeatable deployment approach using Infrastructure as Code.

Instead of manually creating AWS resources through the AWS Management Console, Terraform scripts define and manage the entire infrastructure in a declarative manner.

2.2 Concept and Design Principles

The concept is based on three main principles:

- **Automation** → Reduce manual work and human error
- **Scalability** → Easily extend infrastructure
- **Security** → Use IAM and controlled access

The website content is stored in Amazon S3, distributed globally via CloudFront, and accessed securely using IAM credentials.

3. Architecture Design

3.1 System Architecture Overview

The architecture follows a cloud-native design where a static website is hosted in S3 and delivered globally using CloudFront.

3.2 Architecture Components

1. **IAM (Identity and Access Management)**
 - Provides secure programmatic access
 - Manages permissions and credentials
2. **Amazon S3**
 - Hosts static website files
 - Provides scalable and durable storage
3. **Amazon CloudFront**
 - Acts as a Content Delivery Network (CDN)
 - Reduces latency by caching content globally

3.3 Architecture Workflow

1. User sends request via browser
2. Request goes to CloudFront
3. CloudFront fetches content from S3
4. Content is cached and delivered globally

4. Technologies Used

4.1 Amazon Web Services (AWS)

- IAM → Security and authentication
- S3 → Static website hosting
- CloudFront → Content delivery

4.2 Terraform (Infrastructure as Code)

Terraform is used to:

- Define cloud resources
- Automate infrastructure provisioning

- Manage infrastructure state
- Ensure reproducibility

4.3 Supporting Tools

- AWS CLI
- Terraform CLI
- Web browser for testing

5. Terraform Project Structure

5.1 File Structure Overview

```
|—— provider.tf
|—— s3.tf
|—— cloudfront.tf
|—— variables.tf
|—— terraform.tfvars
|—— outputs.tf

|—— index.html
|—— README.md
```

5.2 provider.tf

Defines the AWS provider and region configuration.

5.3 s3.tf

Handles:

- S3 bucket creation
- Public access configuration
- Static website hosting
- File upload

5.4 cloudfront.tf

Defines: CloudFront distribution , CDN configuration , HTTPS access

5.5 variables.tf

Declares variables such as: AWS region , Bucket name , Project name

5.6 terraform.tfvars

Stores actual values for variables, making the project reusable.

5.7 outputs.tf

Displays:

- CloudFront URL
- S3 website endpoint

6. Implementation and Deployment Process

6.1 Terraform Initialization

terraform init

Initializes the project and installs required providers.

6.2 Planning and Validation

terraform plan

Shows the resources that will be created before applying changes.

6.3 Infrastructure Provisioning

terraform apply

Creates:

- S3 bucket
- CloudFront distribution
- Policies and configurations

6.4 Website Deployment

- Upload index.html to S3

- Configure S3 for static hosting
- Access via CloudFront URL

7. Testing and Validation

The infrastructure was tested to ensure reliability and reproducibility.

Testing Steps:

1. Run:

terraform apply

→ Infrastructure created successfully

2. Access website via CloudFront URL
→ Website loads correctly

3. Run:

terraform destroy

→ Infrastructure removed

4. Re-run:

terraform apply

→ Infrastructure recreated successfully

Result:

The system is fully **automated, reproducible, and testable**, fulfilling IaC principles.

8. Improvements from Phase 1 & Phase 2

- Refactored Terraform into multiple files
- Introduced variables instead of hardcoding
- Improved IAM configuration
- Optimized CloudFront setup
- Added outputs for better monitoring
- Improved code readability

9. Challenges and Solutions

9.1 IAM Configuration Issues

Problem: Authentication errors
Solution: Fixed credentials and permissions

9.2 S3 Public Access Issues

Problem: Website not accessible
Solution: Adjusted public access settings

9.3 CloudFront Deployment Delay

Problem: Slow deployment
Solution: Waited for global propagation

9.4 Terraform Debugging

Problem: Syntax and dependency errors
Solution: Used terraform plan and logs

10. Results and Final Evaluation

The project successfully achieved:

- Automated deployment
- Secure configuration
- Global accessibility
- Scalable architecture
- Clean Terraform structure

The system can be reused and extended easily.

11. Conclusion

This project demonstrates how Infrastructure as Code can simplify cloud deployment while improving scalability, security, and automation.

Using AWS and Terraform, a fully functional and globally accessible static website was deployed. The project highlights the importance of automation and modern DevOps practices in cloud computing.